

ECG-Scope: A Complete, Beginner-Friendly Guide

How an AI Reads an Electrocardiogram, Explained From Zero

Saim Zafar

Abstract

This document explains, from absolutely no assumed knowledge, how our project takes a recording of a heartbeat and predicts which of five heart-condition categories it belongs to, while also showing *why* it made that decision. We start with what an electrocardiogram (ECG) even is, build up the idea of turning it into numbers, introduce neural networks and convolutions gently with everyday analogies and worked numerical examples, walk through the exact model we used layer by layer, explain how it learns, how it explains itself (Grad-CAM), how we measured whether it actually works, and how well it transfers to data from hospitals it never saw. We then walk through the serving code (`ml.py`) line by line with examples, describe the deployed web application, and end with an honest list of limitations and a glossary. Throughout, the guiding principle is: every idea is first told as a plain-language story, then made precise. This is a research and educational project; it is **not** a medical device and must never be used for real diagnosis.

Contents

1	Introduction	3
1.1	What you will be able to understand by the end	3
2	What Is an ECG?	3
2.1	The shape of a single heartbeat	3
3	The Five Categories	4
3.1	Why HYP and STTC overlap (an important subtlety)	4
4	Turning an ECG Into Numbers	4
5	Machine Learning in One Page	5
6	Neural Networks and Convolutions	5
6.1	The core problem: recognising shapes	5
6.2	A filter: an automatic shape-finder	5
6.3	The word "convolution"	6
7	The Model: ECGNet, Layer by Layer	6
7.1	Stacking filters into blocks	6
7.2	What is inside each block	6
7.3	Following one ECG through the assembly line	6
7.4	Summarising the whole heartbeat	7
7.5	The classifier: making the decision	7

7.6 The code	7
8 Input Normalization	8
9 Making a Prediction	8
10 Training: How the Model Learns	8
10.1 The learning loop	9
10.2 The loss function	9
10.3 Gradient descent and backpropagation	9
10.4 Epochs, batches, and augmentation	9
11 Explaining the Decision: Grad-CAM	9
11.1 The workflow as a story	9
11.2 A full worked example	10
11.3 The math, compactly	10
12 Measuring Success: Evaluation	10
12.1 Why AUC and not accuracy	10
12.2 Guards against cheating	10
12.3 Results	11
13 Cross-Dataset Generalization and AdaBN	11
13.1 Harmonising labels across datasets	11
13.2 The results	11
13.3 The key finding	11
14 The Serving Code: ml.py Walkthrough	12
14.1 Setup	12
14.2 <code>analyze</code> – the manager	12
14.3 <code>_prep</code> – clean the signal	12
14.4 <code>load_model</code> – fetch the brain once	12
14.5 <code>_predict_one</code> – the real work	12
14.6 <code>analyze_batch</code> – the AdaBN feature	13
15 The Product (System Architecture)	13
16 Limitations and Disclaimer	13
17 Summary	14
18 The Heartbeat Waveform and What Each Class Looks Like	14
18.1 The visual signature of each category	14
19 A Single Filter, Computed by Hand	15
20 A Miniature Forward Pass with Numbers	15
21 Reading the App’s Output	16

22 The Data Pipeline	16
23 Why These Design Choices	16
24 Frequently Asked Questions	17
25 Glossary	18

1 Introduction

Heart disease is one of the most common causes of illness in the world, and one of the oldest, cheapest tools for spotting it is the *electrocardiogram*, or ECG. An ECG is a recording of the small electrical signals your heart produces every time it beats. Trained doctors learn to read the squiggly lines of an ECG and recognise patterns that hint at specific problems.

The goal of this project was to build a computer program that can do a version of that reading automatically: you give it a 12-lead ECG, and it tells you which of five broad categories the ECG most likely falls into, *and* it highlights the parts of the heartbeat that drove its answer so a human can sanity-check it. Around that core we built a complete, usable website with accounts and history, and we ran a small research study into how well such a model survives when it meets ECGs recorded at hospitals it never trained on.

This guide is written for someone with no background in machine learning or medicine. We will build every concept slowly. If at any point a term is unfamiliar, the Glossary (Section 25) defines it in one line.

1.1 What you will be able to understand by the end

- What an ECG is and how it becomes a grid of numbers.
- What the five categories (NORM, MI, STTC, CD, HYP) mean.
- What a neural network and a "convolution" actually are.
- Exactly how our model turns an ECG into a diagnosis, step by step.
- How the model learned to do this.
- How it explains its decisions (Grad-CAM).
- How we measured success, and how it generalises to new hospitals.
- How the real code and the website fit together.

2 What Is an ECG?

Every heartbeat starts with a tiny burst of electricity inside the heart. That electricity tells the heart muscle when to squeeze. Because the human body conducts electricity, that signal spreads out and can be measured at the skin as a very small voltage (a tiny electrical "pressure", measured in millivolts, thousandths of a volt).

An ECG machine has sticky pads called *electrodes* placed on the body. Each measurement viewpoint is called a *lead*. A *12-lead* ECG looks at the heart's electricity from twelve different angles at the same time, which gives a much fuller picture than a single viewpoint, just as photographing an object from twelve angles tells you more than one snapshot.

2.1 The shape of a single heartbeat

A normal heartbeat, viewed on an ECG, is not a random wiggle. It has a recognisable sequence of features that always appear in the same order:

- a small bump (the **P wave**) as the upper chambers activate,
- a tall, sharp spike (the **QRS complex**) as the main pumping chambers fire,
- another bump (the **T wave**) as the heart electrically "recovers" before the next beat.

Almost everything a doctor (or our model) looks for is a change in the shape, size, or timing of these features. For example, an unusually tall QRS spike can hint at a thickened heart muscle, and an abnormal T wave can hint at a problem with the recovery phase.

3 The Five Categories

Our model sorts each ECG into five broad diagnostic *superclasses*. Each one is actually an umbrella covering many finer diagnoses, but here is what each means in plain language.

NORM (Normal). The ECG looks healthy: a regular rhythm and normal wave shapes. This is the baseline everything else is compared against.

MI (Myocardial Infarction). A heart attack: blood flow to part of the heart muscle was blocked, injuring it. On the ECG this leaves marks such as abnormal Q waves and ST/T shifts. In short, evidence the muscle was damaged by a blood-supply problem.

STTC (ST/T Change). An abnormality in the "recovery" part of the beat (the ST segment and T wave). It can signal reduced blood flow, chemical imbalances, drug effects, or strain. It is a broad "recovery looks wrong" bucket.

CD (Conduction Disturbance). A problem in the heart's electrical "wiring", the system that carries the signal in the right order and timing. Examples include bundle branch blocks (which widen the QRS spike) and AV blocks (which delay the signal). The beat still happens, but the signal travels abnormally.

HYP (Hypertrophy). Thickened heart muscle, usually from years of strain such as high blood pressure. Because there is more muscle, the electrical signals are larger, so the ECG shows taller waves, often with secondary recovery (ST/T) changes.

3.1 Why HYP and STTC overlap (an important subtlety)

Thickened muscle (HYP) commonly *causes* recovery-phase changes (STTC) as a side effect, a combination clinicians call "LVH with strain." So the two often appear on the same ECG. In our data, about half of the HYP cases were also labelled STTC. This is why our model frequently calls HYP cases "STTC": it sees the more obvious recovery change and reports that, missing the deeper cause. It is a genuine, well-known difficulty, not a simple bug. A single ECG can belong to several categories at once, which is why this is a *multi-label* problem.

4 Turning an ECG Into Numbers

A computer cannot look at a wiggly line; it can only work with numbers. So the first real step is understanding how an ECG becomes numbers.

The machine does not record continuously. Instead it takes rapid *snapshots* of the voltage. In our project it samples each lead **100 times per second** for **10 seconds**, giving **1000 readings per lead**. Each reading is a single number, such as 0.12 millivolts.

With **12 leads** recorded at once, one ECG becomes a grid (a table) of numbers:

$$\mathbf{X} \in \mathbb{R}^{1000 \times 12}.$$

That is, 1000 rows (one per time snapshot) and 12 columns (one per lead). Each cell holds one voltage reading. The familiar wavy line you see on screen is simply *one column* of this grid plotted in order.

Example. If lead II (one of the twelve) reads $[-0.10, -0.08, 0.05, 0.80, 0.20, -0.05]$ over six snapshots, the big jump to 0.80 is the QRS spike (the main beat), and the surrounding small values are the quieter parts of the beat.

We trained on **PTB-XL**, a public dataset of roughly 21,000 real 12-lead ECGs, each labelled by cardiologists, using the 100 Hz version of the recordings.

5 Machine Learning in One Page

Before the model, here is the single most important idea, stripped of all jargon.

A machine-learning *model* is just a **function: numbers go in, numbers come out**. Our model takes the grid of ECG numbers in, and produces **5 numbers** out, one score per category. The biggest score becomes the diagnosis. Everything technical that follows is only about *how* the box turns the input numbers into *good* output numbers.

Crucially, nobody writes medical rules into the box by hand. The box contains thousands of internal numbers called *parameters* (or “weights”). At the start these are random, so the box guesses nonsense. *Training* (Section 10) is the process of slowly adjusting those numbers until the box’s outputs match the correct answers on thousands of examples. After training, those numbers encode everything the model “knows.”

6 Neural Networks and Convolutions

Our model is a particular kind of function called a **neural network**, and more specifically a **convolutional neural network (CNN)**. Let us build up what that means.

6.1 The core problem: recognising shapes

To judge a heart, you have to recognise meaningful *shapes* in the signal: a spike here, a dip there, an abnormal bump. A heart problem reveals itself by changing these shapes. So the model needs a way to scan the signal and find shapes automatically.

6.2 A filter: an automatic shape-finder

The tool a CNN uses for this is a **filter**. A filter is a small pattern-detector that slides along the signal looking for one specific shape. The best everyday comparison is the “Find” feature (Ctrl+F) in a document: you type a word, and the computer scans the whole text and marks every place the word appears. A filter does the same, but it searches for a *shape* in the ECG instead of a *word* in text.

Worked example. Imagine a row of numbers that are just heights of the ground as you walk:

1, 1, 2, 8, 2, 1, 1, 1, 2, 9, 2, 1

There are two *spikes* (the 8 and the 9). A spike is the pattern “low, then HIGH, then low.” Now build a tiny machine that finds spikes. Take a window 3 numbers wide and slide it along:

- positions 1–3: 1, 1, 2, slowly rising, no spike, score **low**.
- positions 2–4: 1, 2, 8, rising, score medium.
- positions 3–5: 2, 8, 2, low–HIGH–low, that is a spike! score **high**.
- ...continue sliding to the end ...

The output is a new row of scores whose *high* values land exactly where the spikes were. That sliding machine is a filter. In the real model the numbers are ECG voltages, the window is 7 readings wide, and there are 32 different filters, each hunting a different little shape. Critically, the filters are *not* told which shapes to look for; they discover useful shapes during training.

6.3 The word "convolution"

The mathematical name for "slide this filter across and record the matches" is a *convolution*. That is all the scary word means. In code it is one layer: `nn.Conv1d`. The "1D" means the data is a one-dimensional signal over time (a line), not a two-dimensional image.

7 The Model: ECGNet, Layer by Layer

Our model is called ECGNet. It is small by design, about **47,000 parameters**. We explain it in the order data flows through it, then show the code.

7.1 Stacking filters into blocks

A complex heart pattern cannot be spotted in one go, so filters are stacked into three *blocks*, where each block builds on the one below. Picture a team of workers passing reports up a chain:

- **Block 1 (32 filters)** scans the raw signal for *simple* shapes (spikes, dips, slopes) and reports where each appears.
- **Block 2 (64 filters)** does *not* look at the raw signal. It reads Block 1's reports and detects *combinations* of simple shapes (e.g. "a spike then a dip").
- **Block 3 (128 filters)** reads Block 2's reports and detects still larger, more meaningful patterns, the kind that begin to look like diagnostic features.

This is why the count grows $32 \rightarrow 64 \rightarrow 128$: deeper blocks track more, and richer, patterns.

7.2 What is inside each block

Each block applies four operations in order:

1. **Convolution** (`Conv1d`): the sliding filters.
2. **Batch Normalization** (`BatchNorm1d`): rescales the filter outputs so values stay in a stable range. This makes training reliable and is central to the generalization study (Section 13).
3. **ReLU**: a simple rule that keeps positive values and turns negatives into zero. This lets the network represent relationships more complex than straight lines.
4. **Max-Pooling** (`MaxPool1d`): keeps the strongest response in each small region and discards the rest, halving the length of the signal.

7.3 Following one ECG through the assembly line

It helps to track the *size* of the data as it flows, written as (channels, length):

Stage	Size	Meaning
Input ECG (after flip)	(12, 1000)	12 leads, 1000 readings each
After Block 1	(32, 500)	32 shape-maps, length halved by pooling
After Block 2	(64, 250)	64 combined patterns
After Block 3	(128, 125)	128 deep patterns
After global pooling	(128)	one summary number per pattern
After classifier	(5)	five category scores

Notice the pattern: the signal gets *shorter* (1000→500→250→125) but *deeper* (12→32→64→128). It trades fine timing detail for a richer understanding of which shapes are present.

7.4 Summarising the whole heartbeat

After the three blocks, pattern information is spread across many time positions. A **global average pooling** step collapses the time axis: for each of the 128 deep patterns it keeps a single number describing how strongly that pattern appeared anywhere in the whole heartbeat. The model thus moves from "what is happening at each instant" to "what is present in this ECG overall," producing 128 summary numbers.

7.5 The classifier: making the decision

Those 128 numbers feed a small **classifier**: two fully connected layers, $128 \rightarrow 64 \rightarrow 5$. A fully connected layer simply mixes all its inputs together with learned weights. The classifier weighs the summary and outputs the 5 category scores. It is the part that learned associations such as "when these patterns are strong, the ECG is likely MI."

7.6 The code

```
class ECGNet(nn.Module):
    def __init__(self, num_leads=12, num_classes=5):
        super().__init__()
        self.conv_block1 = nn.Sequential(
            nn.Conv1d(num_leads, 32, 7, padding=3),
            nn.BatchNorm1d(32), nn.ReLU(), nn.MaxPool1d(2))
        self.conv_block2 = nn.Sequential(
            nn.Conv1d(32, 64, 5, padding=2),
            nn.BatchNorm1d(64), nn.ReLU(), nn.MaxPool1d(2))
        self.conv_block3 = nn.Sequential(
            nn.Conv1d(64, 128, 3, padding=1),
            nn.BatchNorm1d(128), nn.ReLU(), nn.MaxPool1d(2))
        self.global_pool = nn.AdaptiveAvgPool1d(1)
        self.classifier = nn.Sequential(
            nn.Linear(128, 64), nn.ReLU(),
            nn.Dropout(0.3), nn.Linear(64, num_classes))

    def forward(self, x, return_features=False):
        x = x.permute(0, 2, 1)          # (batch, time, leads) -> (batch, leads, time)
        x = self.conv_block1(x)
        x = self.conv_block2(x)
        f = self.conv_block3(x)          # kept for Grad-CAM
        out = self.classifier(self.global_pool(f).squeeze(-1))
        return (out, f) if return_features else out
```


The numbers in the code match the table above exactly: `Conv1d(12,32,7)` is "12 leads in, 32 filters out, window 7"; `padding=3` pads the edges so the window can reach the ends; `MaxPool1d(2)` halves the length. The `permute` line is pure bookkeeping to match the shape `Conv1d` expects. `return features` optionally hands back `f`, the deep feature maps Grad-CAM needs.

8 Input Normalization

Before the signal enters the network it is **normalized**, scaled to a standard size. Different machines record at different "loudness", and the model was trained on a standard scale, so a new ECG must be put on the same scale or the model gets confused.

For each lead with readings x_t , mean μ , and standard deviation (spread) σ :

$$x'_t = \frac{x_t - \mu}{\sigma + \varepsilon},$$

with a tiny ε to avoid dividing by zero.

Worked example showing why it matters. Suppose two machines record the *same* heartbeat, one "quiet", one "loud":

	Machine A (quiet)	Machine B (loud)
Raw readings	2, 4, 6, 8, 10	4, 8, 12, 16, 20
Subtract mean	-4, -2, 0, 2, 4 ($\mu = 6$)	-8, -4, 0, 4, 8 ($\mu = 12$)
Divide by spread	-1.41, -0.71, 0, 0.71, 1.41	-1.41, -0.71, 0, 0.71, 1.41

The two recordings started with completely different numbers but end *identical*. Normalization erases the "how loud" difference and keeps only the *shape*, so the model sees every ECG on the same footing.

9 Making a Prediction

The model outputs 5 *raw scores* (called logits), e.g. [2.1, -1.5, 0.3, -2.0, -3.1]. These are not yet probabilities. Because an ECG can have several conditions at once, each score is squashed independently into a 0–1 probability with the **sigmoid** function:

$$p_c = \frac{1}{1 + e^{-z_c}}.$$

Our example becomes roughly {NORM 0.89, MI 0.18, STTC 0.57, CD 0.12, HYP 0.04}. The highest, NORM at 0.89, is reported as the headline diagnosis, while all five probabilities are shown to the user.

10 Training: How the Model Learns

Nothing above is hand-programmed with medical rules. The model *learns* which shapes and patterns matter by being corrected on labelled examples, over and over.

10.1 The learning loop

1. **Start random.** Every filter looks for a random shape; predictions are guesses.
2. **Use the answer key.** Each PTB-XL ECG has a correct label from cardiologists.
3. **Show one ECG, get a guess** (5 probabilities).
4. **Measure how wrong it was** with a *loss* (next subsection).
5. **Nudge every parameter** a tiny step in the direction that reduces the loss.
6. **Repeat** for thousands of ECGs, about 20 full passes through the data.

An everyday analogy: learning to shoot a basketball. You shoot, see you missed left, adjust slightly right, and repeat hundreds of times, never given a formula, only feedback, until you score reliably.

10.2 The loss function

For a prediction $\mathbf{p}$ and true labels $\mathbf{y}$ (each $y_c \in \{0, 1\}$), we use *binary cross-entropy*:

$$\mathcal{L} = - \sum_{c=1}^5 \left[y_c \log p_c + (1 - y_c) \log(1 - p_c) \right].$$

In words: if the true label is 1 and the model said a high probability, the loss is small; if it said a low probability, the loss is large. The loss is a single number measuring "how wrong", small is good.

10.3 Gradient descent and backpropagation

"Nudge every parameter in the direction that reduces the loss" is done by *gradient descent*. For each parameter the model computes how a small change would change the loss (the *gradient*), then steps slightly the way that lowers it. *Backpropagation* is the efficient bookkeeping that computes all these gradients in one sweep backwards through the network. Each step is tiny, but millions of them accumulate into a model that is usually right.

10.4 Epochs, batches, and augmentation

Training processes the data in small groups called *batches*, and one full pass over all the data is an *epoch* (we used about 20). To make the model robust, we also apply random **augmentations** during training, small added noise, dropping a lead, gentle baseline drift, slight amplitude scaling, so it sees varied versions of each beat and does not over-rely on quirks of the training set. We save the version of the model that scored best on a held-out validation set (not necessarily the last one), which guards against *overfitting* (memorising the training data instead of learning general patterns).

11 Explaining the Decision: Grad-CAM

A prediction you cannot question is of little use, so the model also shows *where it looked*. We use **Grad-CAM**, adapted to 1D signals.

11.1 The workflow as a story

Think of a student who just answered a question, and you say "highlight the parts that made you choose that." Grad-CAM gets the model to do exactly that:

1. The model gives its answer (say STTC).
2. We ask it to justify that answer.
3. It reviews all the internal patterns it noticed in the heartbeat.
4. It works out which of those patterns mattered most for STTC.
5. It finds where those patterns were active in time.
6. It marks those moments, strongest brightest, as the red overlay on the trace.

11.2 A full worked example

Shrink the real thing (128 patterns over 125 time-points) to a toy: 3 patterns over 4 time-points. Suppose the activations (how active each pattern was at each time) are:

P1: [0, 2, 0, 0]	(fires at time 2)
P2: [1, 1, 1, 1]	(mild everywhere)
P3: [0, 0, 0, 3]	(fires at time 4)

The model predicted STTC, and the backward step gives an importance per pattern: $w = [0.9, 0.1, -0.5]$ (P1 strongly supports STTC, P2 barely matters, P3 argues against).

1. **Scale each pattern by its importance:**
 $0.9 \cdot [0, 2, 0, 0] = [0, 1.8, 0, 0]$; $0.1 \cdot [1, 1, 1, 1] = [0.1, 0.1, 0.1, 0.1]$; $-0.5 \cdot [0, 0, 0, 3] = [0, 0, 0, -1.5]$.
2. **Add them up per time-point:** $[0.1, 1.9, 0.1, -1.4]$.
3. **ReLU** (negatives $\rightarrow 0$): $[0.1, 1.9, 0.1, 0]$. Time 4 had strong activity, but from a pattern that argued *against* STTC, so it is correctly dropped.
4. **Scale to 0–1** (divide by max 1.9): $[0.05, 1.0, 0.05, 0]$.

So time-point 2 is the bright-red hot spot. Interpretation: "the model chose STTC mainly because of what happened at time 2, where an STTC-supporting pattern fired." In the real model the curve comes out 125 long and is stretched back to 1000 points so it lines up under the trace.

11.3 The math, compactly

With $A_{k,t}$ the activation of pattern k at time t and y_c the predicted score:

$$\alpha_k = \frac{1}{T} \sum_t \frac{\partial y_c}{\partial A_{k,t}}, \quad \text{CAM}_t = \text{ReLU}\left(\sum_k \alpha_k A_{k,t}\right).$$

12 Measuring Success: Evaluation

12.1 Why AUC and not accuracy

Plain accuracy ("what fraction did it get right") is misleading when classes are imbalanced: if 90% of ECGs are normal, a model that always says "normal" is 90% accurate but useless. We instead use **AUC** (Area Under the ROC Curve). AUC measures how well the model *rank*s a positive case above a negative one: 0.5 is random guessing, 1.0 is perfect. It does not depend on choosing a cutoff and is robust to imbalance. **Macro-AUC** is the AUC computed per class and then averaged, so every category counts equally.

12.2 Guards against cheating

- **Patient-disjoint splits:** the test ECGs come from patients the model never saw in training, so it cannot win by memorising a person. We verified zero patient overlap.

- **Multiple seeds:** we retrained three times with different random starts to confirm the result was stable, not a lucky run.

12.3 Results

Class	Test AUC
NORM	0.944
MI	0.924
STTC	0.933
CD	0.910
HYP	0.901
Macro-AUC	0.920 $\pm$ 0.002

This is competitive with published results for this kind of model on PTB-XL.

13 Cross-Dataset Generalization and AdaBN

A model that works on its own data may fail on data from a different hospital, because different machines, populations, and labelling conventions shift the statistics. We tested this directly.

13.1 Harmonising labels across datasets

Different datasets use different labelling schemes. To compare fairly, we mapped each dataset’s diagnostic codes (SNOMED-CT codes) onto our five superclasses. This mapping is careful manual work, and it revealed, for instance, that some datasets contain very few examples of certain classes.

13.2 The results

Setting	Macro-AUC
PTB-XL (in-domain)	0.920
Chapman-Shaoxing (unseen, no adaptation)	0.876
Georgia (unseen, no adaptation)	$\approx$ 0.59
Georgia + test-time BatchNorm adaptation (AdaBN)	0.87

13.3 The key finding

Performance dropped sharply on a truly unseen hospital. Investigation showed that much of the drop was *not* a fundamental failure but a mismatch in the **Batch Normalization** statistics stored inside the network: those were estimated on PTB-XL and are wrong for a new source. **AdaBN** (Adaptive Batch Normalization) fixes this without any retraining or labels: we recompute those statistics from a batch of the new hospital’s *unlabelled* signals, after which performance recovers close to the in-domain level. Standard input normalization and augmentation alone did *not* close the gap, and a class-weighting experiment also failed to help, both honest negative results that pointed us toward the real cause. The deployed product applies AdaBN when a user uploads a batch of ECGs.

14 The Serving Code: `ml.py` Walkthrough

The website's "brain" is a file called `ml.py`. Its one job: ECG in $\rightarrow$ diagnosis plus explanation out. We follow a single ECG through it.

14.1 Setup

```
CLASSES = ["NORM", "MI", "STTC", "CD", "HYP"]    # output order
MODEL_PATH = ".../assets/best_model.pt"          # trained weights
DEVICE = torch.device("cpu")                     # one ECG at a time
NORMALIZE = True                                 # model expects normalized input
```

These are just settings the rest of the file reads.

14.2 `analyze` – the manager

```
def analyze(sig):
    return _predict_one(load_model(), _prep(sig))
```

Read inside-out: clean the ECG (`_prep`), fetch the model (`load_model`), then predict (`_predict_one`). `analyze` itself does no real work; it just orders the helpers, like a front-desk handing your form to the right departments.

14.3 `_prep` – clean the signal

```
def _prep(sig):
    sig = np.asarray(sig, dtype=np.float32)
    if NORMALIZE:
        sig = (sig - sig.mean(0)) / (sig.std(0) + 1e-8)
    return sig
```

This is the normalization from earlier: subtract each lead's mean, divide by its spread, so every ECG is on the standard scale.

14.4 `load_model` – fetch the brain once

```
_model = None
def load_model():
    global _model
    if _model is None:
        m = ECGNet()
        m.load_state_dict(torch.load(MODEL_PATH, weights_only=True))
        m.eval(); _model = m
    return _model
```

A fresh `ECGNet()` is an empty brain; `load_state_dict` pours in the trained numbers; `eval()` sets prediction mode. The `if _model is None` check loads it only the first time and reuses it thereafter.

14.5 `_predict_one` – the real work

```
def _predict_one(model, sig):
    x = torch.tensor(sig).unsqueeze(0)
    probs = torch.sigmoid(model(x))[0]
    top = int(probs.argmax())
    return {
        "probabilities": {CLASSES[i]: float(probs[i]) for i in range(5)},
        "predicted_class": CLASSES[top],
        "importance": _grad_cam(model, sig, top),
        "disclaimer": DISCLAIMER,
    }
```

Run the model to get 5 raw scores; `sigmoid` turns them into probabilities; `argmax` picks the highest as the diagnosis; `_grad_cam` builds the red heatmap; everything is packed into a dictionary the website draws.

14.6 analyze_batch – the AdaBN feature

```
def analyze_batch(signals):
    m = copy.deepcopy(load_model())
    for mod in m.modules():
        if isinstance(mod, nn.BatchNorm1d):
            mod.reset_running_stats(); mod.momentum = None
    prepped = [_prep(s) for s in signals]
    m.train()
    with torch.no_grad():
        m(torch.tensor(np.stack(prepped))) # re-learn stats from user's batch
    m.eval()
    return [_predict_one(m, s) for s in prepped]
```

When a user uploads several ECGs, this makes a throwaway copy of the model, wipes its BatchNorm statistics, runs the user’s batch through to re-learn those statistics from *their* data, then predicts. This is the AdaBN finding turned into a live feature; it adapts to a new source with no retraining and no labels.

15 The Product (System Architecture)

The model is wrapped in a deployed web application:

- **Frontend** (React, hosted on Vercel): the landing page, login/signup, and the analyze page that draws the trace, the Grad-CAM overlay, and the probability bars. It runs in the user’s browser and contains no model code.
- **Backend** (FastAPI, hosted on Hugging Face Spaces): receives an uploaded ECG over the internet, calls `ml.py`, and returns the result. `main.py` is the “front desk”; `ml.py` is the “brain”.
- **Supabase** (database + authentication): manages user accounts and stores each user’s history of analyses, protected so each user only sees their own.

The flow for one analysis:

browser → backend (`main.py`) → `ml.py` (load, analyze, Grad-CAM) → result → browser draws it.

16 Limitations and Disclaimer

This is a **research and educational** project. It is *not* a medical device, has not been clinically validated, and must never be used for diagnosis or treatment decisions. Known limitations:

- **HYP is unreliable:** the model rarely predicts hypertrophy as the top class, usually labelling those cases STTC, because hypertrophy commonly produces ST/T changes.
- **Cross-source accuracy drops:** on ECGs from sources unlike the training data, accuracy falls (partly recoverable with AdaBN on a batch).
- **Ordinary mistakes:** a 0.92-AUC model still misclassifies a meaningful fraction of individual ECGs. AUC measures ranking ability, not perfect per-case correctness.
- **Not novel research:** the techniques used are established; this is a strong portfolio and learning project, not a new scientific contribution.

17 Summary

The pipeline end to end:

voltages $\rightarrow$ 1000×12 grid $\rightarrow$ filters find shapes $\rightarrow$ blocks build patterns $\rightarrow$ summary $\rightarrow$ classifier $\rightarrow$ 5 scores.

Training provides the knowledge, the sigmoid turns scores into probabilities, Grad-CAM provides the explanation, evaluation (patient-disjoint, multi-seed, macro-AUC) confirms it works, and AdaBN provides robustness to new data sources. The whole thing is served through a React frontend and a FastAPI backend with Supabase accounts.

18 The Heartbeat Waveform and What Each Class Looks Like

To understand what the model is hunting for, it helps to know the parts of a single normal heartbeat in a little more detail. In order, each beat shows:

- **P wave** – a small rounded bump as the upper chambers (atria) activate.
- **PR interval** – the short flat gap between the P wave and the spike; it reflects the time the signal takes to travel from the atria to the ventricles.
- **QRS complex** – the tall, narrow spike as the main pumping chambers (ventricles) fire. Its width and height matter a great deal.
- **ST segment** – the short stretch immediately after the spike; normally flat.
- **T wave** – a bump as the ventricles electrically recover.

Diagnosis is, in large part, asking whether each of these is the right size, shape, and timing.

18.1 The visual signature of each category

NORM

Regular spacing, a normal-width QRS, a flat ST segment, and an upright T wave. Nothing stands out.

MI

Often a deep, abnormal "Q" notch at the start of the spike (a sign of old damage), and/or a shifted ST segment (raised or depressed) and inverted T waves in the acute phase. The exact leads affected hint at *where* in the heart the damage is.

STTC

A depressed or elevated ST segment and/or flattened or inverted T waves, the "recovery" portion of the beat looks wrong, without necessarily the Q-wave damage marks of MI.

CD

A widened or oddly shaped QRS spike (bundle branch block) or a stretched PR interval (AV

block). The beat occurs, but the spike’s shape or timing is abnormal because the signal took a slow or wrong path.

HYP

Unusually *tall* QRS spikes (more muscle produces bigger voltages), frequently with secondary ST/T ”strain” changes, which is exactly why HYP and STTC are so easily confused.

This is why a model that finds and measures shapes (Section on filters) is a sensible approach: the shapes are literally the diagnostic evidence.

19 A Single Filter, Computed by Hand

We described a filter as a sliding shape-detector. Here is precisely what ”score how well the window matches” means, with real arithmetic.

A filter is just a short list of numbers (its *weights*), learned during training. To score a window, you multiply each weight by the matching reading, add them up, and add a small constant (the *bias*). This single number is the filter’s response at that position. Then ReLU keeps it if positive, else sets it to zero.

Example. Take a tiny filter of width 3 with weights $w = [-1, 2, -1]$. This particular shape responds strongly to a ”peak” (a value higher than its neighbours), because it adds twice the middle and subtracts the sides. Slide it over the signal $[1, 1, 5, 1, 1]$:

- Window $[1, 1, 5]$: $(-1)(1) + (2)(1) + (-1)(5) = -4$. $\text{ReLU} \rightarrow 0$.
- Window $[1, 5, 1]$: $(-1)(1) + (2)(5) + (-1)(1) = 8$. $\text{ReLU} \rightarrow 8$. **Strong response: a peak is centred here.**
- Window $[5, 1, 1]$: $(-1)(5) + (2)(1) + (-1)(1) = -4$. $\text{ReLU} \rightarrow 0$.

The output map is $[0, 8, 0]$, which correctly lights up exactly where the peak is. A different filter (different weights) would respond to a different shape. The model has 32 such filters in the first block, and *it learns the weights itself* so that the shapes it detects are the ones that help diagnose. The ”padding” in the code simply adds zeros at the edges so the window can be centred on the very first and last readings too.

20 A Miniature Forward Pass with Numbers

Let us run a deliberately tiny example all the way through, to see numbers turn into a decision. Suppose (unrealistically small) we have one lead of length 6, one filter, and a two-class problem.

1. **Input:** $x = [0.0, 0.1, 0.9, 0.2, 0.0, 0.1]$ (a single spike at position 3).
2. **Filter** $w = [-1, 2, -1]$ slides across and (after ReLU) produces a feature map that is large near the spike, say $[0, 1.5, 0, 0]$ after pooling.
3. **Summary (global pool):** average the map $\rightarrow$ one number, e.g. 0.375.
4. **Classifier:** multiply that summary by learned weights for each class and add biases, e.g. class A score = 2.0, class B score = -0.5 .
5. **Sigmoid:** $p_A = 1/(1 + e^{-2.0}) \approx 0.88$, $p_B = 1/(1 + e^{0.5}) \approx 0.38$.
6. **Decision:** class A wins (0.88).

The real model does exactly this, just with 12 leads, 1000 readings, 224 filters across three blocks, 128 summary numbers, and a $128 \rightarrow 64 \rightarrow 5$ classifier. The principle is identical: detect shapes, summarise, weigh, squash to probabilities, pick the largest.

21 Reading the App’s Output

When you analyse an ECG in the web app, you see four things. Here is how to read them.

The trace

The black line is the ECG signal (lead II), drawn from the voltage numbers.

The red overlay

The Grad-CAM heatmap: redder regions are the moments that most influenced the model’s decision. On a clean beat it tends to sit on the QRS spikes.

The diagnosis

The single highest-probability category, shown large.

The probability bars

All five categories with their probabilities. Because this is *multi-label*, these do *not* have to add up to 100%; each is an independent ”how likely is this one.” Two can both be high if the ECG genuinely shows two things.

Judging confidence. If the top bar is far above the others (say 0.9 versus 0.1), the model is confident. If the top two are close (say 0.55 and 0.45), the model is unsure, and the result should be treated with extra caution. This is common on noisy or borderline ECGs, and it is honest behaviour: a good model expresses doubt rather than faking certainty.

22 The Data Pipeline

A few words on how raw ECGs become training-ready arrays, since this is where much of the careful work lives.

PTB-XL stores each recording in the **WFDB** format: a small header file (`.hea`) describing the signal, plus the signal data itself. A master table (`ptbx1.database.csv`) lists every record with its diagnostic codes, and a second table (`scp.statements.csv`) maps each fine-grained code to one of our five *superclasses*. Our loading code:

1. reads each record’s raw signal (we use the 100 Hz version, giving the 1000×12 grid),
2. converts the listed diagnostic codes into a 5-long *multi-hot* label (a 1 for each superclass present, else 0),
3. splits the data using PTB-XL’s official ”stratified fold” column: folds 1–8 for training, fold 9 for validation, fold 10 for testing. Because the folds keep all of a patient’s records together, the test patients are never seen in training.

For the cross-dataset study, other databases (Chapman-Shaoxing, Georgia) use **SNOMED-CT** diagnostic codes; we wrote a mapping from those codes onto the same five superclasses, resampled their 500 Hz signals down to 100 Hz, and matched the lead order, so every dataset could be fed to the same model in the same format.

23 Why These Design Choices

A natural question is ”why build it this particular way?” Brief, honest answers:

Why a CNN?

ECG diagnosis is about local shapes that can appear anywhere along the signal. Convolutions are exactly the tool for ”find this shape wherever it occurs,” so they fit the problem naturally.

Why 1D?

An ECG is a signal over time (a line), not a 2D image, so the filters slide in one direction (time).

Why a *small* model (47k parameters)?

PTB-XL is moderately sized, and a small model trains fast on free hardware, is less prone to overfitting, and is enough to reach competitive accuracy. Bigger is not automatically better.

Why kernel sizes 7, 5, 3?

Wider windows early capture broader local shapes; narrower windows deeper combine already-summarised features. This coarse-to-fine pattern is a common, effective default.

Why sigmoid + binary cross-entropy, not softmax?

Because an ECG can have several conditions at once (multi-label). Softmax forces the probabilities to compete and sum to one (good for "pick exactly one"); sigmoid lets each class be judged independently.

Why macro-AUC?

It is robust to class imbalance and threshold-free, and averaging per class means rare categories are not drowned out by common ones.

Why normalize inputs?

To remove device-to-device scale differences so the model sees every ECG on the same footing.

Why AdaBN?

Because the cross-hospital gap turned out to be largely a BatchNorm-statistics mismatch, and AdaBN fixes that cheaply, at test time, without labels.

24 Frequently Asked Questions

Does the model "understand" hearts?

No. It has learned statistical patterns that correlate with the labels cardiologists assigned. It has no concept of a heart; it maps numbers to scores in a way that happened to work on the training data.

Is 0.92 AUC the same as 92% accuracy?

No. AUC measures ranking ability across all possible thresholds; accuracy is the fraction correct at one chosen threshold. They are different numbers and AUC is the more informative one here.

Why did a built-in sample get misclassified?

The model is not perfect; it misses a fraction of cases, especially subtle ones like STTC and HYP. A wrong case is expected behaviour for a model below 100% accuracy, not a malfunction.

Why can it not detect HYP well?

Hypertrophy is largely about signal amplitude and usually co-occurs with ST/T changes, so the model tends to report the more visible STTC instead.

Can I trust it for a real diagnosis?

No. It is a research and educational tool only, not a validated medical device.

What does "multi-label" mean again?

One ECG can belong to several categories at once, so the model outputs an independent probability for each, rather than choosing exactly one.

Where does the "knowledge" live?

In the roughly 47,000 trained numbers stored in `best_model.pt`. An untrained model with random numbers knows nothing.

Why is there a separate training and serving code?

Training (on Kaggle) *creates* the model; serving (`ml.py` on the backend) *uses* it. They run at different times in different places.

What is AdaBN in one sentence?

Re-estimating the model's internal normalization statistics from a batch of the new data, so it

adapts to a new source without retraining or labels.

Could the accuracy be pushed higher?

A little, with more data sources, a bigger model, or ensembling, but gains are modest and perfection is not achievable for this problem.

Is this novel research?

No. The methods are established. It is a strong portfolio and learning project, and a careful empirical study, but not a new scientific contribution.

What is the single most important safeguard in the evaluation?

Patient-disjoint splits: the test patients never appear in training, so the model cannot cheat by memorising individuals.

25 Glossary

ECG

Electrocardiogram; a recording of the heart's electrical activity.

Lead

One viewpoint/angle of the ECG; a 12-lead ECG has twelve.

Sample / snapshot

One voltage reading taken at one instant.

Model

A function that maps input numbers to output numbers.

Parameter / weight

One of the internal numbers a model adjusts during training.

Neural network

A model built from layers of simple operations with learnable weights.

CNN

Convolutional Neural Network; uses sliding filters as its main building block.

Filter / convolution

A small sliding shape-detector; scans the signal and scores matches.

Channel

One filter's output map (or one input lead).

ReLU

An operation that keeps positives and zeroes negatives.

Pooling

Shrinking a signal by summarising small regions (max or average).

Logit

A raw output score before being turned into a probability.

Sigmoid

A function squashing a score into a 0–1 probability.

Loss

A number measuring how wrong a prediction is.

Gradient descent

The method of nudging weights to reduce the loss.

Backpropagation

The efficient way to compute all gradients at once.

Epoch

One full pass through the training data.

Batch

A small group of examples processed together.

Overfitting

Memorising the training data instead of learning general patterns.

AUC

A score for how well a model ranks positives above negatives (0.5 random, 1.0 perfect).

Grad-CAM

A method that highlights which inputs most influenced a prediction.

BatchNorm

A layer that rescales activations to a stable range.

AdaBN

Recomputing BatchNorm statistics on new data to adapt a model without retraining.

Multi-label

A setting where one example can belong to several classes at once.